

BUSINESS PROCESS DOCUMENT

(BPD / Standard Operating Procedure)

End-to-End Encrypted Password Manager

Document Type:	Process Definition Document / SOP
Project Repository:	https://github.com/Aabhi-k/passman
Live Application:	https://passman.letsgetthis.tech/
Classification:	Internal — Engineering & Operations
Version:	1.0
Date:	September 2026
Document Owner:	Engineering Lead / Product Manager

TABLE OF CONTENTS

1. Document Control
2. Purpose & Scope
3. Process Overview
4. Roles & Responsibilities
5. Detailed Process Workflows
 - 5.1 User Registration & Authentication
 - 5.2 Password Vault Management (CRUD)
 - 5.3 Client-Side Encryption Workflow
 - 5.4 Session & Security Management
 - 5.5 System Operations & Monitoring
 - 5.6 Build & Deployment
6. Data Flow Diagram
7. Key Performance Indicators (KPIs)
8. Risk Register & Mitigation
9. Compliance & Security Standards
10. Appendices

1. DOCUMENT CONTROL

This section establishes the governance, ownership, and revision tracking for the Passman Business Process Document.

Field	Details
Document Title	Passman Business Process Document (BPD)
Document Owner	Engineering Lead / Product Manager
Author	Technical Documentation Team
Review Cycle	Quarterly or upon major release
Approval Authority	CTO / Engineering Director
Distribution	Engineering, DevOps, Security, QA, Support

Revision History

Version	Date	Author	Description
1.0	Sept 2026	Tech Docs	Initial BPD covering auth, encryption, CRUD, build, and deployment workflows

2. PURPOSE & SCOPE

2.1 Purpose

This document formally defines the end-to-end business processes, technical workflows, security protocols, and operational procedures for **Passman** — a cross-platform password manager that ensures credentials are protected via client-side AES-GCM encryption and never transmitted or stored in plaintext.

2.2 Scope

- **User Lifecycle:** Registration, authentication, session management, and account settings
- **Password Vault Lifecycle:** Creation, retrieval, update, and deletion of encrypted password entries
- **Encryption Workflow:** Client-side key derivation, AES-GCM encryption/decryption, and secure storage
- **System Operations:** Backend API services, database transactions, JWT security, and error handling
- **Build & Deployment:** Local development, Docker containerization, desktop builds (Tauri), and production deployment
- **Cross-functional Responsibilities:** Frontend, Backend, DevOps, Security, and QA teams

2.3 Out of Scope

- Third-party password import/migration tools
- Browser extension development
- Mobile native application (iOS/Android) workflows
- Enterprise SSO / SAML integration

3. PROCESS OVERVIEW

Passman operates on a **zero-knowledge architecture** where:

1. The **client application** (React + Tauri desktop) performs all encryption/decryption using the **Web Crypto API**
2. The **backend** (Spring Boot) acts as a secure storage layer for encrypted blobs, handling only authentication and metadata
3. The **database** (MySQL) stores only ciphertext, IVs, authentication tags, and user account metadata — never plaintext passwords

3.1 High-Level Process Flow

```
End User (Desktop/Web) → React + Tauri (Frontend) → Web Crypto API (AES-GCM Enc) ↓  
Encrypted Blob (data + iv + authTag) ↓ Spring Boot (Backend) – JWT + Spring Security ↓  
MySQL 8.0 (Persistence)
```

4. ROLES & RESPONSIBILITIES

Role	Responsibilities
End User	Registers account, creates master password, manages password vault, maintains device security
Frontend Engineer	React component development, Tauri integration, encryption module (CryptoUtils), UI/UX, routing
Backend Engineer	Spring Boot API development, JWT implementation, database schema, security configuration, DTO design
DevOps Engineer	Docker/Docker Compose management, environment configuration, CI/CD pipelines, production deployment
Security Engineer	Encryption algorithm review, JWT secret rotation, penetration testing, CORS policy audit
QA Engineer	End-to-end testing of encryption flows, auth regression, cross-platform desktop build validation
Database Admin	MySQL instance management, backup scheduling, performance tuning, user data retention

5. DETAILED PROCESS WORKFLOWS

5.1 PROCESS 1: USER REGISTRATION & AUTHENTICATION

Objective: Enable users to create secure accounts and authenticate via JWT tokens to access their encrypted password vault.

Trigger: User launches Passman application and selects 'Sign Up' or 'Log In'.

Actors: End User, Frontend (React), Backend (Spring Boot), MySQL Database

Registration Process Steps

Step	Activity	System / Actor	Inputs	Outputs
1	Navigate to Sign Up page	End User	—	Sign Up form rendered
2	Enter Username, Email, Password, Confirm Password	End User	Form data	Validated input
3	Validate password match and format	React Frontend	User input	Validation status
4	Send POST /api/auth/register	React + Axios	UserDto	HTTP request
5	Validate email uniqueness	Spring Boot + JPA	Email query	Boolean check
6	Hash password (BCrypt) and generate encryptionSalt	Spring Security	Plaintext password	Hashed password + salt
7	Persist new User entity	Hibernate/JPA	User object	Saved record
8	Generate JWT access token	JwtService	User details	LoginResponseDto (token, userId, salt)
9	Store token in localStorage	React (jwt.js)	JWT string	Persisted session
10	Derive AES key from password + salt (PBKDF2)	Frontend (js)	Password, salt	AES-GCM key
11	Redirect to Dashboard	React Router	Auth state	/dashboard route

Login Flow (Sub-process)

1. User enters Email + Password on Login page
2. Frontend sends POST /api/auth/login with LoginDto
3. Backend authenticates via CachedUserDetailsService
4. Backend validates credentials against BCrypt hash
5. Backend returns JWT token + user metadata (including encryptionSalt)
6. Frontend derives AES key from password + salt
7. Frontend fetches encrypted passwords via GET /api/passwords/getall/{userId}
8. Frontend decrypts each entry client-side for display

Exception Handling

Exception	Cause	Resolution
401 Unauthorized	Invalid credentials	Display error; prompt re-entry
409 Conflict	Email already registered	Prompt user to log in instead
403 Forbidden	Expired/invalid JWT	Redirect to login; clear localStorage
Network timeout	Backend unreachable	Retry logic; offline error message

5.2 PROCESS 2: PASSWORD VAULT MANAGEMENT (CRUD)

Objective: Allow authenticated users to securely create, read, update, and delete password entries, with all encryption handled client-side.

Data Entities: Frontend Plaintext: { website, username, password, notes, category } | Backend Storage: Password entity { id, encryptedData, iv, authTag, user_id, createdAt, updatedAt }

CREATE Process Steps

Step	Activity	Actor / System	Details
1	Click 'Add Password'	End User	Opens InsertPassword form
2	Fill Site Name, URL, Username, Password, Notes	End User	Optional: use built-in password generator
3	Serialize password object to JSON	React	{ "site": "...", "username": "...", "password": "..." }
4	Encrypt JSON using AES-GCM (256-bit)	CryptoUtils.js	Generates IV and authTag; uses derived AES key
5	Send POST /api/passwords/insert	Axios	PasswordDto with encryptedData, iv, authTag, userId
6	Validate JWT via JwtAuthenticationFilter	Spring Security	Extracts and validates Bearer token
7	Persist encrypted blob to passwords table	JPA/Hibernate	Links to User via @ManyToOne
8	Return 201 Created	PasswordController	Success confirmation
9	Refresh Dashboard list	React	Re-fetches and decrypts all entries

READ Process Steps

- Dashboard mounts and calls GET /api/passwords/getall/{userId}
- Backend returns list of encrypted password records
- Frontend iterates records, decrypts each encryptedData using stored AES key + IV + authTag
- Frontend renders decrypted site names, usernames (passwords masked by default)
- User clicks 'reveal' to view plaintext password (decrypted on-demand)

UPDATE Process Steps

- User selects password entry and clicks 'Edit'
- Frontend decrypts selected entry for editing
- User modifies fields
- Frontend re-encrypts modified JSON with NEW IV
- Frontend sends PUT /api/passwords/update/{passwordId}
- Backend updates record with new encryptedData, iv, authTag, and updatedAt

DELETE Process Steps

- User selects entry and clicks 'Delete'
- Frontend shows confirmation dialog
- On confirm, sends DELETE /api/passwords/delete/{passwordId}
- Backend verifies ownership via JWT user context
- Backend deletes record; returns success
- Frontend removes entry from local state

Exception Handling

Exception	Cause	Resolution
Decryption failure	Wrong AES key or corrupted data	Log error; show 'Unable to decrypt — check master password'
404 Not Found	Password ID doesn't exist	Refresh list from server
403 Forbidden	User doesn't own password entry	Reject operation; audit log
Concurrent modification	Update on stale data	Optimistic locking or last-write-wins with timestamp check

5.3 PROCESS 3: CLIENT-SIDE ENCRYPTION WORKFLOW

Objective: Ensure plaintext passwords never leave the user's device by performing all cryptographic operations in the frontend using the Web Crypto API.

Encryption Architecture

```
User Master Password + encryptionSalt (from server) | ▼ PBKDF2 Key Derivation (100,000+ iterations) | ▼ AES-GCM 256-bit Key (stored in AesKeyStore.js) | ▼ Plaintext JSON: { "password": "secret123", ... } | ▼ AES-GCM Encrypt | ██████████ ▼
```

```
▼ ▼ encryptedData iv authTag (Base64) (Base64) (Base64) ■ ■ ■
```

```
██████████ ■ ▼ POST to /api/passwords/insert
```

Key Management

Aspect	Implementation
Key Derivation	PBKDF2 from master password + per-user encryptionSalt
Key Storage	In-memory only (AesKeyStore.js); never persisted to disk
Algorithm	AES-GCM-256 via Web Crypto API
IV Generation	Cryptographically random 12-byte IV per encryption operation
Auth Tag	128-bit GCM authentication tag for integrity verification
Key Lifecycle	Derived on login; cleared on logout / session expiry

Security Controls

- **Zero-Knowledge Proof:** Server cannot decrypt user data; only stores ciphertext
- **Per-Record IV:** Each password entry uses a unique IV to prevent pattern analysis
- **Memory Safety:** AES key exists only in JavaScript memory; cleared on app close
- **No Plaintext Logging:** Frontend and backend logs must never contain decrypted passwords

5.4 PROCESS 4: SESSION & SECURITY MANAGEMENT

JWT Lifecycle

Phase	Activity	Details
Issuance	Upon successful login/register	Signed HS256 JWT with userId, email, expiration
Storage	localStorage (frontend)	Key: passman_token
Transmission	Axios request interceptor	Attached as Authorization: Bearer <token>
Validation	JwtAuthenticationFilter	Validates signature, expiry, and user existence
Refresh	Not implemented (stateless)	User must re-login after expiry
Revocation	Client-side only	Clear localStorage on logout

Spring Security Configuration

- **Public Endpoints:** /api/auth/** (login, register, change-password)
- **Protected Endpoints:** /api/passwords/**, /api/users/** — require valid JWT
- **CORS:** Configured for APP_ORIGIN_URL (frontend origin)
- **Password Encoding:** BCryptPasswordEncoder
- **UserDetailsService:** CachedUserDetailsService with JPA-backed user lookup

Password Change Workflow

Authenticated user navigates to Settings

User provides current password and new password

Frontend sends POST /api/auth/change-password

Backend verifies current password against BCrypt hash

Backend updates password hash in users table

CRITICAL: User must re-derive AES key with new password + existing salt

All existing encrypted entries remain accessible (same salt, new key derivation)

■■ **Note:** Changing the master password does NOT re-encrypt existing vault entries in this version. The encryption key is derived from the password + salt, so a password change effectively changes the decryption key. The current implementation requires users to maintain awareness of this limitation or the system must re-encrypt all entries post-password-change (future enhancement).

5.5 PROCESS 5: SYSTEM OPERATIONS & MONITORING

Health Checks

Component	Endpoint / Method	Frequency
Spring Boot App	GET /actuator/health	Every 30s (Docker healthcheck)
MySQL Database	mysqladmin ping	Every 30s (Docker healthcheck)

Database Schema (Logical)

Table: users

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID
name	VARCHAR(255)	Username
email	VARCHAR(255)	Unique, Not Null
password	VARCHAR(255)	BCrypt hash
encryption_salt	VARCHAR(255)	Not Null

Table: passwords

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID
encrypted_data	LONGTEXT / CLOB	Not Null
iv	VARCHAR(255)	Not Null
auth_tag	VARCHAR(255)	Not Null
user_id	VARCHAR(36)	FK → users.id
created_at	TIMESTAMP	Auto
updated_at	TIMESTAMP	Auto

Logging Requirements

Log Category	Content	Retention
Auth Events	Login success/failure, registration, password changes	90 days
API Access	Endpoint, userId, timestamp, response status	30 days
Security	JWT validation failures, unauthorized access attempts	1 year

Log Category	Content	Retention
Errors	Stack traces, exception messages	30 days
Encryption	Never log plaintext, keys, or decrypted data	N/A

5.6 PROCESS 6: BUILD & DEPLOYMENT

5.6.1 Local Development Setup

Backend:

```
cd backend/ # Prerequisites: Java 17+, Maven 3.6+, MySQL 8.0 mvn clean install #
Configure env.properties (DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET, JWT_EXPIRATION,
APP_ORIGIN_URL) mvn spring-boot:run # Server starts on :8080
```

Frontend:

```
cd frontend/ # Prerequisites: Node.js 16+, npm, Rust + Cargo (for Tauri) npm install npm
run dev # Web dev server npm run tauri dev # Desktop dev mode
```

5.6.2 Docker Deployment

```
cd backend/ # Set environment variables: JWT_SECRET, JWT_EXPIRATION, APP_ORIGIN_URL
docker-compose up -d
```

Service	Image	Ports	Dependencies
app	Custom (OpenJDK 17)	8080:8080	mysql
mysql	mysql:8.0	3307:3306	—

5.6.3 Production Build (Desktop)

```
cd frontend/ npm run tauri build # Outputs: Windows (.msi/.exe), macOS (.dmg/.app),
Linux (.deb/.AppImage)
```

5.6.4 Frontend Web Deployment

- Vercel configuration present (vercel.json)
- Build command: vite build
- Environment variable: VITE_API_BASE_URL pointing to backend

6. DATA FLOW DIAGRAM

```

■■■■■■■■■■ End User ■■■■■■■■■■ ▼
■■■■■■■■■■ 1. INPUT: Master Password ■■■ 2. INPUT:
Password Entry ■■■■■■■■■■ ■■■■■■■■■■
■■■■■■■■■■ ▼▼ ■■■■■■■■■■ Auth ■■■
Encryption ■■■ Module ■■■ (Web Crypto) ■■■■■■■■■■ ■■■ ▼▼
■■■■■■■■■■ JWT Token ■■■ Encrypted Blob ■■■ (Bearer)
■■■ (data+iv>tag) ■■■■■■■■■■ ■■■■■■■■■■ ■■■
■■■■■■■■■■ ▼ ■■■■■■■■■■ Axios HTTP Call ■■■
(Interceptors) ■■■■■■■■■■ ▼ ■■■■■■■■■■ Spring Boot API
■■■ (Port 8080) ■■■■■■■■■■ ▼ ■■■■■■■■■■ MySQL 8.0 ■■■
(Port 3306/3307) ■■■■■■■■■■

```

7. KEY PERFORMANCE INDICATORS (KPIs)

Metric	Target	Measurement
API Response Time	< 200ms (p95)	Spring Boot Actuator / APM
Encryption/Decryption Time	< 50ms per entry	Browser DevTools performance
Login Success Rate	> 99%	Auth logs analysis
Uptime (Backend)	> 99.9%	Docker healthchecks / uptime monitor
Password Vault Load Time	< 1s for 100 entries	Frontend performance timing
Security Incidents	0	Security audit logs
Build Success Rate	> 98%	CI/CD pipeline metrics

8. RISK REGISTER & MITIGATION

Risk	Likelihood	Impact	Mitigation
User forgets master password	High	Critical	No recovery possible by design (zero-knowledge). User must maintain back
AES key exposed in memory	Medium	High	Key stored only in JS memory; cleared on logout. Recommend app auto-lo
JWT token theft	Medium	High	Short expiry times; HTTPS only; consider httpOnly cookie migration
Database breach	Low	High	Data is encrypted at rest; attacker gains only ciphertext without keys
Tauri/Native vulnerability	Low	Medium	Keep Tauri and Rust dependencies updated; monitor CVEs
Cross-origin misconfiguration	Low	High	Strict CORS policy; validate APP_ORIGIN_URL

9. COMPLIANCE & SECURITY STANDARDS

Standard	Compliance Approach
Zero-Knowledge Architecture	Server never possesses decryption keys or plaintext
OWASP Top 10	Input validation, parameterized queries (JPA), CSRF protection (stateless JWT), secure headers
Data Minimization	Only encrypted blobs and minimal metadata stored
Encryption at Rest	AES-GCM ciphertext in database; no plaintext storage
Encryption in Transit	TLS 1.2+ for all API communications
Audit Trail	Login events, CRUD operations logged with userId and timestamp

10. APPENDICES

Appendix A: API Endpoint Reference

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Create new user account
POST	/api/auth/login	No	Authenticate and receive JWT
POST	/api/auth/change-password	Yes	Update account password
POST	/api/passwords/insert	Yes	Create new encrypted password entry
GET	/api/passwords/getall/{userId}	Yes	List all user's password entries
GET	/api/passwords/get/{passwordId}	Yes	Retrieve single password entry
PUT	/api/passwords/update/{passwordId}	Yes	Update password entry
DELETE	/api/passwords/delete/{passwordId}	Yes	Delete password entry
GET	/actuator/health	No	System health check

Appendix B: Technology Stack

Layer	Technology	Version
Backend Framework	Spring Boot	3.4.3
Language	Java	17
Security	Spring Security + JWT (jjwt)	0.12.6
ORM	Hibernate / Spring Data JPA	—
Database	MySQL	8.0
Frontend Framework	React	19
Build Tool	Vite	6.x
Desktop Wrapper	Tauri	2.4.1
HTTP Client	Axios	1.8.4
Encryption	Web Crypto API (AES-GCM)	Native
Containerization	Docker + Docker Compose	—

Appendix C: File Structure Reference

```
passman/ ■■■■ backend/ ■ ■■■■ src/main/java/com/aabhi/pasman/ ■ ■ ■■■■ controller/
(Auth, Password, User) ■ ■ ■■■■ service/ (authservice, passwordservice, userservice) ■
■ ■■■■ model/ (User, Password entities) ■ ■ ■■■■ dto/ (auth, password, user DTOs) ■ ■
■■■■ repository/ (JPA repositories) ■ ■ ■■■■ security/ ■ ■ ■ ■■■■ jwt/ (JwtService,
Filter, EntryPoint, Config) ■ ■ ■ ■■■■ web/ (SecurityConfig) ■ ■ ■■■■ exception/
(Global exception handling) ■ ■■■■ src/main/resources/ ■ ■ ■■■■ application.properties
■ ■■■■ Dockerfile ■ ■■■■ docker-compose.yml ■■■■ frontend/ ■ ■■■■ src/ ■ ■ ■■■■
Components/ ■ ■ ■ ■■■■ Home/ ■ ■ ■ ■■■■ Login/ ■ ■ ■ ■■■■ Signup/ ■ ■ ■ ■■■■
Dashboard/ ■ ■ ■ ■■■■ Manager/InsertPassword/ ■ ■ ■ ■■■■ Setting/ ■ ■ ■ ■■■■
Encryption/ (AesKeyStore.js, CryptoUtils.js) ■ ■ ■ ■■■■ api/ (api.js, jwt.js) ■ ■ ■■■■
App.jsx ■ ■ ■■■■ main.jsx ■ ■■■■ src-tauri/ (Rust Tauri config) ■ ■■■■ package.json ■
■■■■ vite.config.js ■■■■ README.md
```

Appendix D: Glossary

Term	Definition
AES-GCM	Advanced Encryption Standard — Galois/Counter Mode; provides authenticated encryption
IV	Initialization Vector; random nonce ensuring unique ciphertext for identical plaintext
Auth Tag	Authentication tag in GCM mode; verifies ciphertext integrity
PBKDF2	Password-Based Key Derivation Function 2; derives cryptographic key from password
JWT	JSON Web Token; compact, URL-safe token for stateless authentication
Tauri	Rust-based framework for building lightweight, secure desktop applications
Zero-Knowledge	Architecture where service provider cannot access user data

11. APPROVAL SIGN-OFF

Role	Name	Signature	Date
Document Author			
Engineering Lead			
Security Officer			
QA Lead			
CTO / Approver			

END OF DOCUMENT